Building a Controller (C++)

Steven Martinez

Introduction

This project implements a cascaded 3D controller for a quadcopter in C++. The logic was adapted from theoretical exercises and respects the NED (North-East-Down) coordinate system. The code includes math protections to handle edge cases like division-by-zero and constraints to ensure flight stability.

1 Implemented Controller

1.1 Motor Commands (`GenerateMotorCommands`)

Translates collective thrust and desired moments (M_x, M_y, M_z) into individual motor commands for an "X" configuration drone. It properly accounts for the drone's arm length $(L/\sqrt{2})$ and the drag/thrust ratio (κ).

```
cmd.desiredThrustsN[0] = (collThrustCmd + momentCmd.x / (L / sqrt(2.f)) + momentCmd.y / (
    L / sqrt(2.f)) - momentCmd.z / kappa) / 4; // front left
cmd.desiredThrustsN[1] = (collThrustCmd - momentCmd.x / (L / sqrt(2.f)) + momentCmd.y / (
    L / sqrt(2.f)) + momentCmd.z / kappa) / 4; // front right
cmd.desiredThrustsN[2] = (collThrustCmd + momentCmd.x / (L / sqrt(2.f)) - momentCmd.y / (
    L / sqrt(2.f)) + momentCmd.z / kappa) / 4; // rear left
cmd.desiredThrustsN[3] = (collThrustCmd - momentCmd.x / (L / sqrt(2.f)) - momentCmd.y / (
    L / sqrt(2.f)) - momentCmd.z / kappa) / 4; // rear right
```

1.2 Body Rate Control (`BodyRateControl`)

A proportional controller that calculates desired 3-axis moments. It accounts for the drone's moments of inertia (I_{xx}, I_{yy}, I_{zz}) to output the correct physical torque.

```
V3F I(Ixx, Iyy, Izz);
V3F error = pqrCmd - pqr;
momentCmd = kpPQR * error * I;
```

1.3 Roll/Pitch Control (`RollPitchControl`)

Converts target lateral accelerations into roll/pitch rates (p and q). It accounts for the NED frame (thrust acts negatively in Z) and constrains the target angles to prevent the drone from flipping.

```
  float acc = -collThrustCmd / mass;
  if (collThrustCmd < 0.001f) { acc = -0.001f / mass; }
  // Elementos de la matriz R
  float R33 = R(2, 2);
  float R21 = R(1, 0);
  float R11 = R(0, 0);
  float R22 = R(1, 1);
  float R12 = R(0, 1);
```

```
float b_x_a = R(0, 2);
float b_y_a = R(1, 2);

// Elementos deseados (con constrain)
float b_x_c = CONSTRAIN(accelCmd.x / acc, -maxTiltAngle, maxTiltAngle);
float b_y_c = CONSTRAIN(accelCmd.y / acc, -maxTiltAngle, maxTiltAngle);

float b_dot_x_c = kpBank * (b_x_c - b_x_a);
float b_dot_y_c = kpBank * (b_y_c - b_y_a);

pqrCmd.x = (R21 * b_dot_x_c - R11 * b_dot_y_c) / R33;
pqrCmd.y = (R22 * b_dot_x_c - R12 * b_dot_y_c) / R33;
```

1.4 Altitude Control (`AltitudeControl`)

A PID controller regulating vertical thrust. An integrator term (`i_term`) is included specifically to handle drones with non-ideal mass (Scenario 4).

```
float b_z = R(2, 2);
velZCmd = CONSTRAIN(velZCmd, -maxAscentRate, maxDescentRate);
float z_error = posZCmd - posZ;
float z_error_dot = velZCmd - velZ;

// 1. ACUMULAR EL ERROR INTEGRAL
integratedAltitudeError += z_error * dt;

float p_term = kpPosZ * z_error;
float d_term = kpVelZ * z_error_dot;
// 2. SUMAR EL TERMINO INTEGRAL (KiPosZ)
float i_term = KiPosZ * integratedAltitudeError;

float u_1_bar = p_term + d_term + i_term + accelZCmd;

float u1 = (9.81f - u_1_bar) / b_z;

thrust = mass * u1;
```

1.5 Lateral Position Control (`LateralPositionControl`)

A PD controller tracking XY positions. Both the target velocities and the output accelerations are constrained to ensure smooth navigation limits.

```
velCmd.x = CONSTRAIN(velCmd.x, -maxSpeedXY, maxSpeedXY);
velCmd.y = CONSTRAIN(velCmd.y, -maxSpeedXY, maxSpeedXY);

float x_error = posCmd.x - pos.x;
float x_error_dot = velCmd.x - vel.x;
float p_term_x = kpPosXY * x_error;
float d_term_x = kpVelXY * x_error_dot;
accelCmd.x += p_term_x + d_term_x;

accelCmd.x = CONSTRAIN(accelCmd.x, -maxAccelXY, maxAccelXY);

float y_error = posCmd.y - pos.y;
float y_error_dot = velCmd.y - vel.y;
float p_term_y = kpPosXY * y_error;
float d_term_y = kpVelXY * y_error_dot;
accelCmd.y += p_term_y + d_term_y;

accelCmd.y = CONSTRAIN(accelCmd.y, -maxAccelXY, maxAccelXY);
```

1.6 Yaw Control (`YawControl`)

A P controller that wraps the angle error between $[-\pi, \pi]$ using modulo arithmetic. This ensures the drone always takes the shortest rotational path to the target heading.

```
float error = fmodf(yawCmd - yaw, 2.f * F_PI);

if (error > F_PI) {
    error -= 2.f * F_PI;
} else if (error < -F_PI) {
    error += 2.f * F_PI;
}
yawRateCmd = kpYaw * error;
```

2 Flight Evaluation

The controller was tested across all required scenarios, successfully passing all evaluation metrics.

Scenario 2: Attitude Control

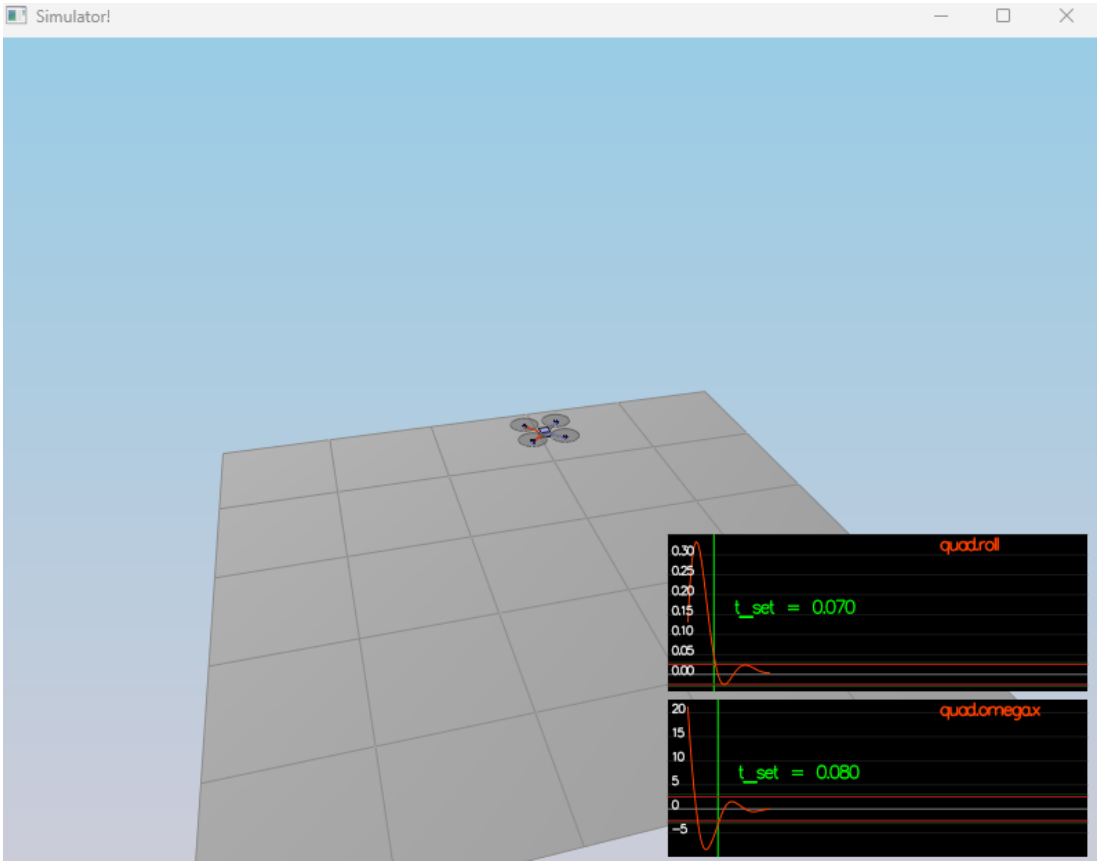

Figure 1: Successful body rate and roll/pitch stabilization.

- `PASS: ABS(Quad.Roll) was less than 0.025000 for at least 0.750000 seconds`
- `PASS: ABS(Quad.Omega.X) was less than 2.500000 for at least 0.750000 seconds`

Scenario 3: Position Control

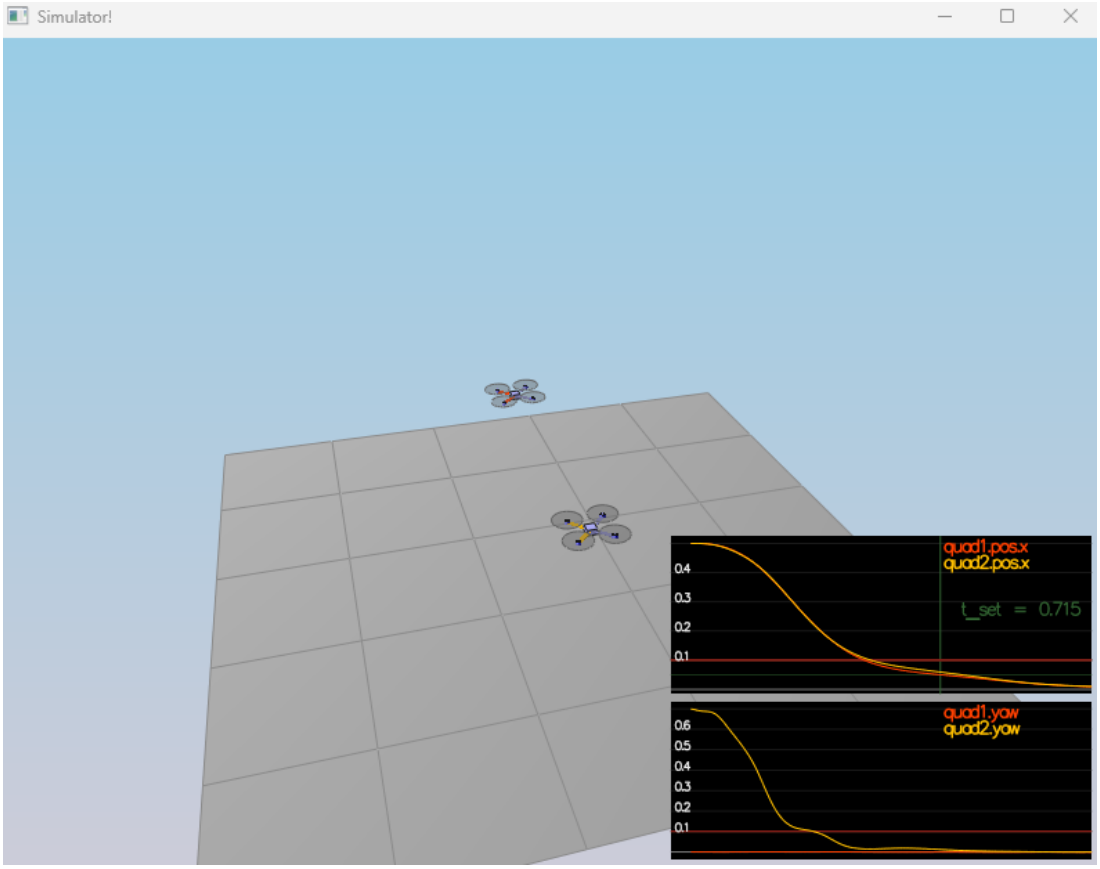

Figure 2: Two quads reaching their target positions and yaw angles.

- `PASS: ABS(Quad1.Pos.X) was less than 0.100000 for at least 1.250000 seconds`
- `PASS: ABS(Quad2.Pos.X) was less than 0.100000 for at least 1.250000 seconds`
- `PASS: ABS(Quad2.Yaw) was less than 0.100000 for at least 1.000000 seconds`

Scenario 4: Non-idealities

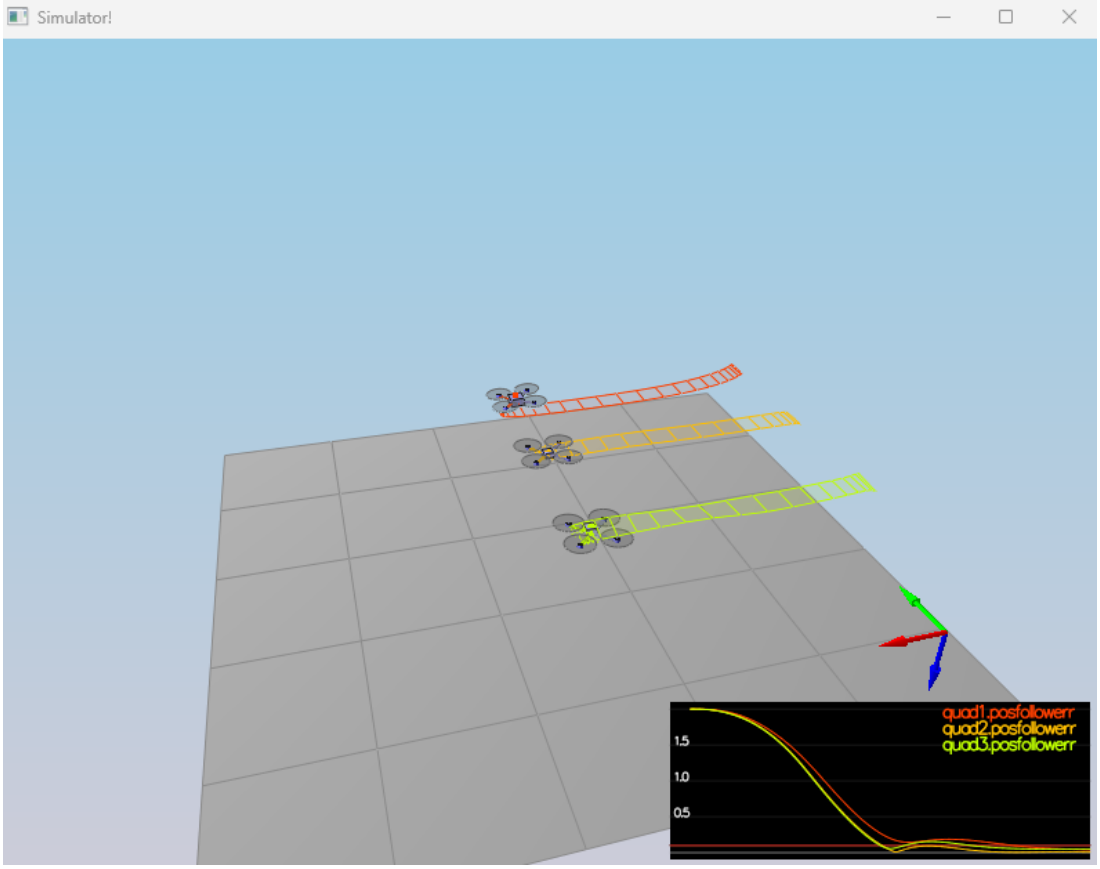

Figure 3: Three quads with different masses reaching the target, managed by the altitude integrator.

- `PASS: ABS(Quad1.PosFollowErr) was less than 0.100000 for at least 1.500000 seconds`
- `PASS: ABS(Quad2.PosFollowErr) was less than 0.100000 for at least 1.500000 seconds`
- `PASS: ABS(Quad3.PosFollowErr) was less than 0.100000 for at least 1.500000 seconds`

Scenario 5: Trajectory Follow

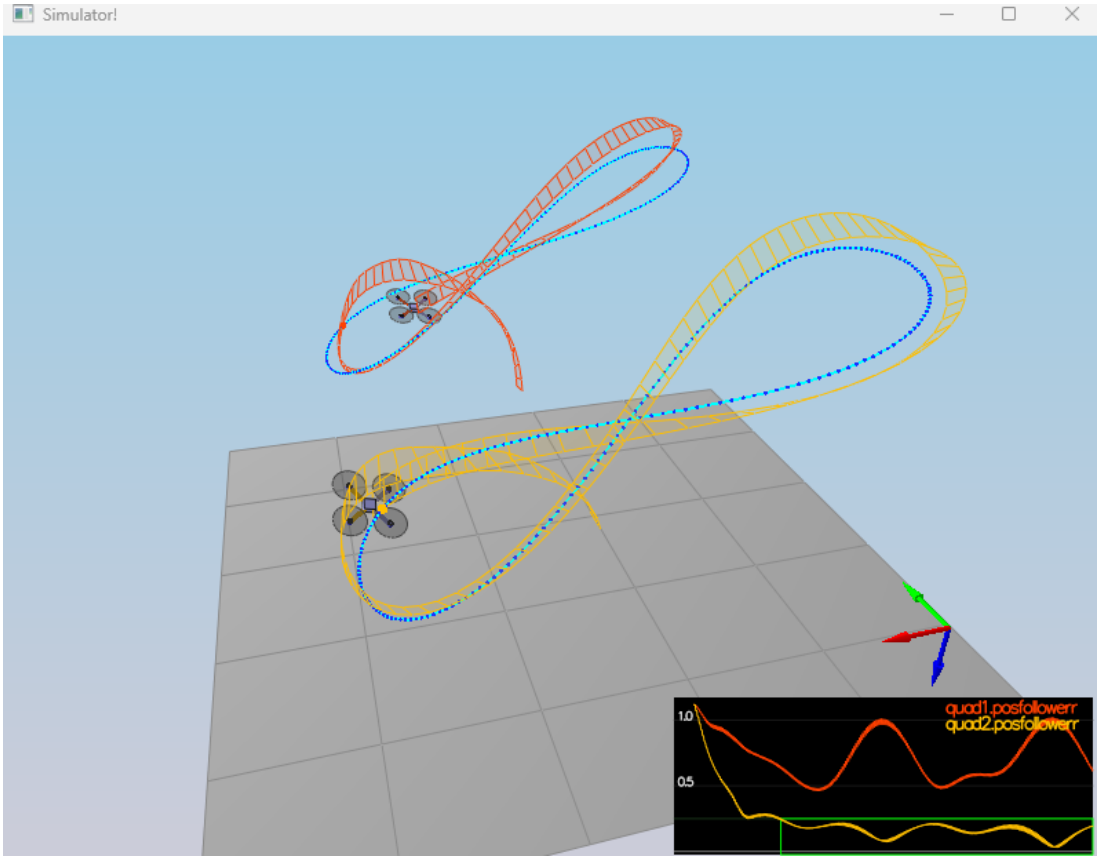

Figure 4: Successful tracking of the Figure-8 trajectory.

- `PASS: ABS(Quad2.PosFollowErr) was less than 0.250000 for at least 3.000000 seconds`